

Performance Testing

Từ JMeter/k6 đến AI-Augmented Workflow.

Xây dựng hệ thống chịu tải thực tế trong kỷ nguyên AI —
Kiểm thử hiệu năng là một quy trình kỹ thuật, không phải
một nút bấm.

Lộ trình seminar

-
- | | | |
|----|----------|-------------------------------------|
| 01 | Bối cảnh | Vì sao hiệu năng là chuyện sống còn |
|----|----------|-------------------------------------|
-
- | | | |
|----|----------|--|
| 02 | Nền tảng | Loại kiểm thử, chỉ số, SLO, Workload Model |
|----|----------|--|
-
- | | | |
|----|--------------|--|
| 03 | Công cụ & AI | Quy trình 4 bước, JMeter, k6, AI sinh script |
|----|--------------|--|
-
- | | | |
|----|--------|------------------------------|
| 04 | Rủi ro | Ba kiểu kịch bản AI sinh sai |
|----|--------|------------------------------|
-
- | | | |
|----|------------|-------------------------------------|
| 05 | Thực chiến | Checklist, ROI của độ trễ, tổng kết |
|----|------------|-------------------------------------|
-

53%

Người dùng rời bỏ website nếu thời gian tải trang vượt quá 3 giây.

CASE STUDY · 11/2022

Ticketmaster sập khi mở bán Eras Tour

Đợt presale tour của Taylor Swift kéo lượng truy cập vượt xa mọi dự báo — Ticketmaster cho biết hệ thống nhận khoảng 3,5 tỷ lượt request trong ngày. Website tê liệt, đợt mở bán công khai bị hủy, vụ việc dẫn tới phiên điều trần trước Thượng viện Mỹ.

Một sự kiện tăng tải đột biến được báo trước hàng tháng — nhưng không được spike test đúng mức.

Ngộ nhận và sự thật

NGỘ NHẬN



10.000 VU

Đồng nhưng phân bố ngẫu nhiên —
không phản ánh hành vi người dùng thật.

SỰ THẬT



60% Tìm kiếm



30% Thêm giỏ



10% Checkout

60 · 30 · 10

Ít VU hơn nhưng đúng tỷ lệ hành vi —
kết quả mới có giá trị kết luận.

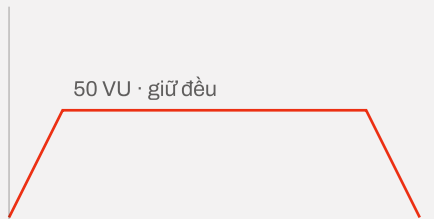
Vị trí trong vòng đời phát triển



Test sớm — sửa rẻ. Gắn performance regression test vào CI/CD để mỗi bản build đều được đo, thay vì chờ đến sát ngày go-live mới biết SQLite nghẽn connection.

Ba loại kiểm thử hiệu năng

LOAD TESTING

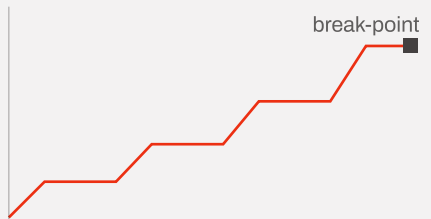


Tải chuẩn của một ngày thường.
Hệ thống có chạy ổn ở mức bình thường?

LOAD · ERP nội bộ, ngân hàng giờ hành chính

SOAK · Dịch vụ chạy 24/7, rò rỉ bộ nhớ

STRESS TESTING

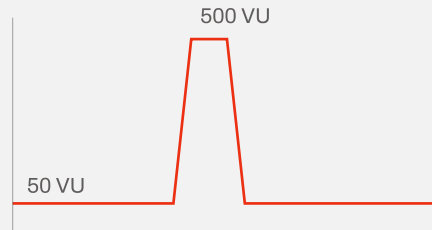


Tăng dần vượt ngưỡng thiết kế.
Hệ thống sập ở mức nào?

STRESS · SaaS tăng trưởng, cần biết trần tải

VOLUME · Hệ thống báo cáo, kho dữ liệu lớn

SPIKE TESTING

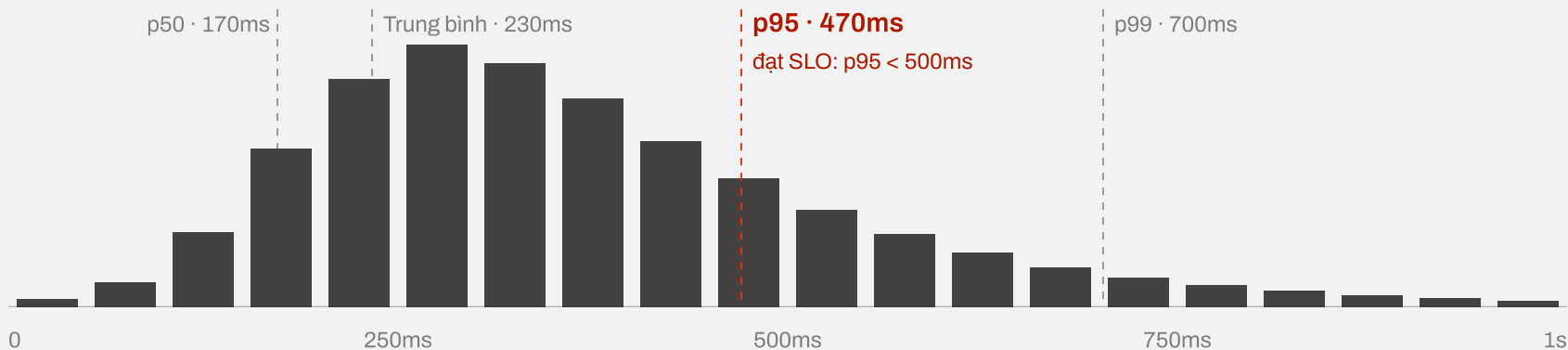


50 → 500 VU trong 30 giây.
Hệ thống chịu được cú sốc đột ngột?

SPIKE · Sàn TMĐT Black Friday, bán vé sự kiện

SCALABILITY · Hạ tầng cloud mở rộng theo tải

Chỉ số cốt lõi và SLO



Trung bình là con số dễ nhìn nhưng dễ đánh lừa: với p95 đạt 470ms, cứ 100 khách thì 5 người đang chờ lâu gấp đôi con số trung bình, và 1 người (p99) chờ gần gấp ba. Ở một sàn thương mại điện tử triệu đơn mỗi ngày, 1% đó là hàng nghìn giao dịch bị bỏ dở. SLO vì vậy luôn đặt trên p95/p99, kèm Error Rate và Throughput (RPS).

Workload Model: mô hình hóa hành vi thật

60% TÌM KIẾM

30% THÊM GIỎ

10% CHECKOUT



≈ 600 request/phút
duyet danh mục, lọc theo giá

≈ 300 lượt
thêm hàng vào giỏ

≈ 100 đơn
nghiep vụ nặng nhất

Ngân hàng số



50% xem số dư · 35% chuyển khoản ·
15% khác

Nền tảng streaming



70% xem · 20% tìm kiếm · 10% tương tác

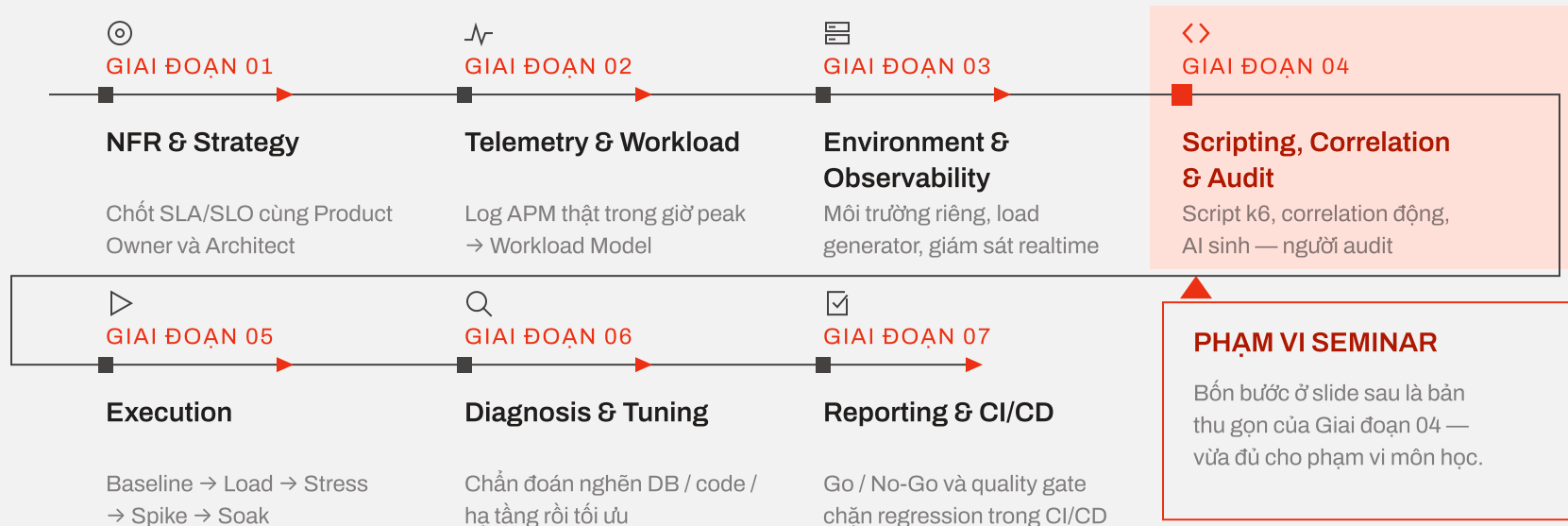
Mạng xã hội



60% lướt · 25% tương tác · 15% đăng bài

Mỗi hệ thống có một nhịp hành vi riêng — Workload Model không thể copy từ hệ thống này sang hệ thống khác.

Bức tranh toàn cảnh: quy trình tại doanh nghiệp



Quy trình bốn bước

Bước 01

Capture & Model

Bắt file HAR và log từ trình duyệt, dựng Workload Model từ hành vi thật.

Bước 02

AI Scripting

▶ Đưa HAR kèm prompt chuẩn cho ChatGPT/Claude để sinh kịch bản k6 hoặc JMeter.

Bước 03

Human Audit

▶ Người rà soát tỷ lệ tải, correlation token, race condition và ngưỡng SLO.

Bước 04

Execute & Verify

▶ Chạy test, thu metrics, đối chiếu SLO và ghi lại mọi failure mode.

- Vòng lặp: mỗi bản build chạy lại bước 04 trong CI/CD để phát hiện hồi quy hiệu năng sớm. HAR — HTTP Archive — là bản ghi toàn bộ request trình duyệt đã gửi đi.

JMeter và k6

Apache JMeter

Giao diện đồ hoạ, cấu hình bằng XML.

Kéo-thả trên GUI, không bắt buộc biết code

Hệ sinh thái plugin lâu đời, rất nhiều mẫu sẵn

Nặng tài nguyên khi mô phỏng tải lớn

Hợp đội QA truyền thống, dự án doanh nghiệp

Grafana k6

Kịch bản JavaScript, chạy bằng dòng lệnh.

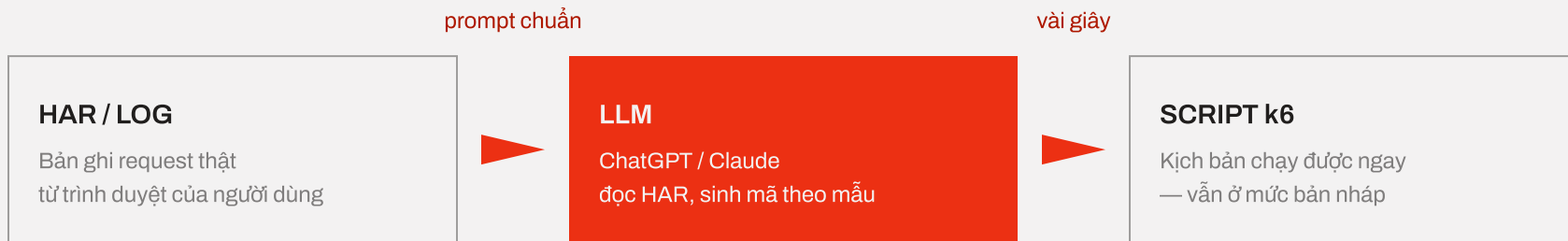
Viết test như viết code, dễ review qua Git

Siêu nhẹ, tốn ít RAM/CPU cho cùng số VU

Cloud-native, cắm thẳng vào CI/CD

Hợp đội dev-centric — lựa chọn của nhóm cho EShop

Hướng AI-Augmented



≈ 2 giờ viết tay → ≈ 30 giây sinh tự động

Tốc độ tăng vượt bậc, nhưng script sinh ra chỉ là bản nháp thô — chưa đảm bảo đúng logic nghiệp vụ hay dữ liệu động.

Đây là lý do bước Audit không thể bỏ qua.

Khi AI sinh kịch bản sai

NGUYÊN NHÂN

Hardcode session token / cart ID. AI đọc HAR rồi dán cứng giá trị, thay vì viết correlation động.

Phân bố tài phi thực tế. AI chia đều 33/33/33% thay vì phân ánh hành vi search-heavy.

Bỏ sót race condition. AI không tự nghĩ ra kịch bản tranh chấp dữ liệu.

Ngưỡng quá tự tin. AI đề xuất threshold không dựa trên baseline nào.

BIỂU HIỆN

Cả 500 VU dùng chung một token và một Cart ID — test “chạy được” nhưng không mô phỏng người dùng độc lập.

Báo cáo SLO “đẹp”, nhưng không mô tả đúng một ngày Black Friday.

Hai người cùng checkout sản phẩm cuối cùng — lỗi không bao giờ lộ ra.

Bài test pass vì ngưỡng được đặt rộng hơn năng lực thật của hệ thống.

CÁCH XỬ LÝ

Kiểm tra script có bóc tách token/cookie/cart ID theo từng VU; nếu không, tự viết lại correlation.

Đối chiếu lại với log và HAR gốc trước khi chạy chính thức.

Bổ sung thủ công các kịch bản tranh chấp vào bộ test.

Luôn benchmark baseline trước khi tin ngưỡng AI đề xuất.

AI giúp bạn viết nhanh hơn, không giúp bạn đúng hơn. Audit là bắt buộc, không phải tùy chọn.

Chiến lược kết hợp

TRUYỀN THỐNG	KẾT HỢP	AI-AUGMENTED
Độ tin cậy cao Kiểm soát hoàn toàn Viết kịch bản chậm	AI draft nhanh Người audit & refine Tool đo chính thức	Sinh script tốc độ cao Tốt cho bản nháp Cần người verify

AI và công cụ truyền thống không cạnh tranh — chúng bổ trợ nhau. Vùng giao chính là quy trình bốn bước: AI draft → con người audit & refine → JMeter/k6 execute và đo lường chính thức.

Checklist thực hành

- Chốt SLO rõ ràng trước khi viết dòng script đầu tiên.
- Đo baseline trước mọi bài spike hoặc stress — không có mốc thì không có kết luận.
- Theo dõi p95 và p99, không dừng lại ở giá trị trung bình.
- Audit 100% kịch bản do AI sinh ra trước khi chạy chính thức.
- Test ngay khi API sẵn sàng, không chờ giao diện hoàn thiện.
- Ghi lại mọi failure mode quan sát được như tài sản kỹ thuật của nhóm.

Cái giá của 100 mili-giây

AMAZON · +100ms thời gian tải



-1% doanh thu

GOOGLE 2006 · +0,5 giây trả kết quả



-20% lưu lượng

0 5% 10% 15% 20%

Đây là lý do SLO p95 < 500ms không phải con số tùy tiện: mỗi 100ms tối ưu được trong bài test là tiền giữ lại cho doanh nghiệp.

Nguồn: Greg Linden (Amazon, trình bày năm 2006) và Marissa Mayer (Google, Web 2.0 Summit 2006)

Ba điều cần nhớ

01

Workload Model đúng quan trọng hơn số VU cao.

Mô phỏng đúng hành vi người dùng, rồi mới nói đến quy mô tải.

02

AI tăng tốc scripting, con người kiểm chứng nghiệp vụ.

Tốc độ là của máy, trách nhiệm về tính thực tế vẫn là của người.

03

Shift-Left: test sớm thì sửa rẻ.

Đo hiệu năng ngay khi có API, và đo lại ở mỗi bản build.

Hỏi & Đáp

Cảm ơn Thầy/Cô và các bạn đã theo dõi phần trình bày của nhóm

Rất mong nhận được câu hỏi và góp ý — đặc biệt về Workload Model và cách nhóm audit kịch bản do AI sinh ra.